

Trimmed SemanticKITTI Dataset Report

1. Overview of the Trimmed Dataset

The `dataset_trimmed` folder contains an optimized, scaled-down version of the SemanticKITTI/KITTI Odometry dataset. To minimize the massive storage and memory footprint of the original 84+ GB dataset, we have extracted **every 10th frame** (reducing the temporal frequency from 10 Hz to 1 Hz).

Data Consistency Mechanism: To ensure that existing standard dataloaders do not crash, the trimming script did not just skip files. Instead, it sequentially renamed the retained files (e.g., original frame 0 became 0, original frame 10 became 1, frame 20 became 2). Crucially, the text files (`poses.txt` and `times.txt`) were correspondingly sliced, so that line 1 in `poses.txt` accurately reflects the pose of the new frame 1. This guarantees 100% data alignment.

2. File and Folder Structure

The dataset follows a rigid directory structure, split into sequential drives (sequences 00 through 21).

```
dataset_trimmed/
├── sequences/
│   ├── 00/
│   │   ├── calib.txt
│   │   ├── poses.txt
│   │   ├── times.txt
│   │   └── velodyne/
│   │       ├── 000000.bin
│   │       ├── 000001.bin
│   │       └── ...
│   │   └── labels/ <-- (Only in sequences 00 to 10)
│   │       ├── 000000.label
│   │       ├── 000001.label
│   │       └── ...
│   ├── 01/
│   └── ... (up to 21)
```

3. Detailed File Explanations & Formats

A. LiDAR Data (`velodyne/*.bin`)

- Format:** Uncompressed binary files containing raw arrays of 32-bit floats (`float32`). Note that this is *not* a standard 3D format like `.pcd` or `.ply` . You must read it programmatically (e.g., using `numpy.fromfile(file, dtype=np.float32).reshape(-1, 4)`).
- Content:** Each `.bin` file is a single 360-degree sweep from a Velodyne HDL-64E scanner. It contains ~100,000 to 120,000 points. Each point is defined by 4 values: `[x, y, z, reflectance]` .
- How to Use:** This represents the raw input features for your model. The $N \times 4$ array will be fed directly into your 3D neural network (like a Sparse Convolutional Network or PointNet) to represent the physical world.

B. Semantic Labels (`labels/*.label`)

- Format:** Uncompressed binary files containing an array of 32-bit unsigned integers (`uint32`).
- Content:** There is exactly one `uint32` value for every point in the corresponding `.bin` file.
 - Lower 16 bits:** Encode the semantic class ID (e.g., 10 = car, 30 = pedestrian).
 - Upper 16 bits:** Encode the instance ID, which differentiates between separate objects of the same class (e.g., Car A vs. Car B).
- How to Use:** These are the **ground-truth targets** for training. During supervised training, the network predicts a class for each point, and the loss is calculated by comparing predictions against these labels. By mapping these IDs to your Indian driving scenarios classes (cars, trucks, pedestrians), you train the model to segment traffic.

C. Ego-Motion Poses (`poses.txt`)

- **Format:** Standard text file. Each line contains 12 space-separated floating-point numbers. Line index i corresponds directly to frame i (e.g., line 5 corresponds to `000005.bin`).
- **Content:** The 12 numbers represent a flattened 3×4 transformation matrix (R for rotation, t for translation). It describes the exact 3D position and orientation of the left camera (and by extension, the car) in the global world coordinate system for that specific frame.
- **How to Use:** LiDAR point clouds are sparse. A common technique is to stitch multiple consecutive `.bin` frames together to make a much denser, detailed point cloud (making cars and pedestrians easier to see). You use the matrices in `poses.txt` to align the past frames perfectly with the current frame's coordinate system.

D. Sensor Calibration (`calib.txt`)

- **Format:** Standard text file containing multiple 3×4 matrices (P_0 , P_1 , P_2 , P_3 , and T_r). It remains static for all frames within a sequence.
- **Content:**
 - T_r (Transformation): The matrix required to convert 3D point coordinates from the LiDAR sensor's frame of reference to the left camera's frame of reference.
 - $P_0 - P_3$: Camera projection matrices describing focal lengths and optical centers.
- **How to Use:** If you are purely training a 3D LiDAR segmentation model, you might not use this file. However, if you decide to download the corresponding RGB camera video and want to colorize your point cloud (or project the 3D points onto the 2D image for sensor fusion), you multiply your 3D points by T_r and then by P_2 (for the left color camera).

E. Timestamps (`times.txt`)

- **Format:** Text file containing one floating-point number per line. Line index i corresponds to frame i .
- **Content:** The timestamp in seconds indicating when the LiDAR sweep was triggered.
- **How to Use:** Useful for motion estimation. By knowing the precise time delta between frames alongside the `poses.txt`, you can calculate the ego-vehicle velocity.

4. Summary

By keeping every 10th frame, you have effectively turned an 84 GB dataset into an ~8.4 GB dataset without losing geographic or environmental diversity. You possess the raw point clouds (input) and the pointwise semantic labels (targets). The data is fully prepared, correctly aligned, and ready to be ingested by a 3D Deep Learning dataloader!